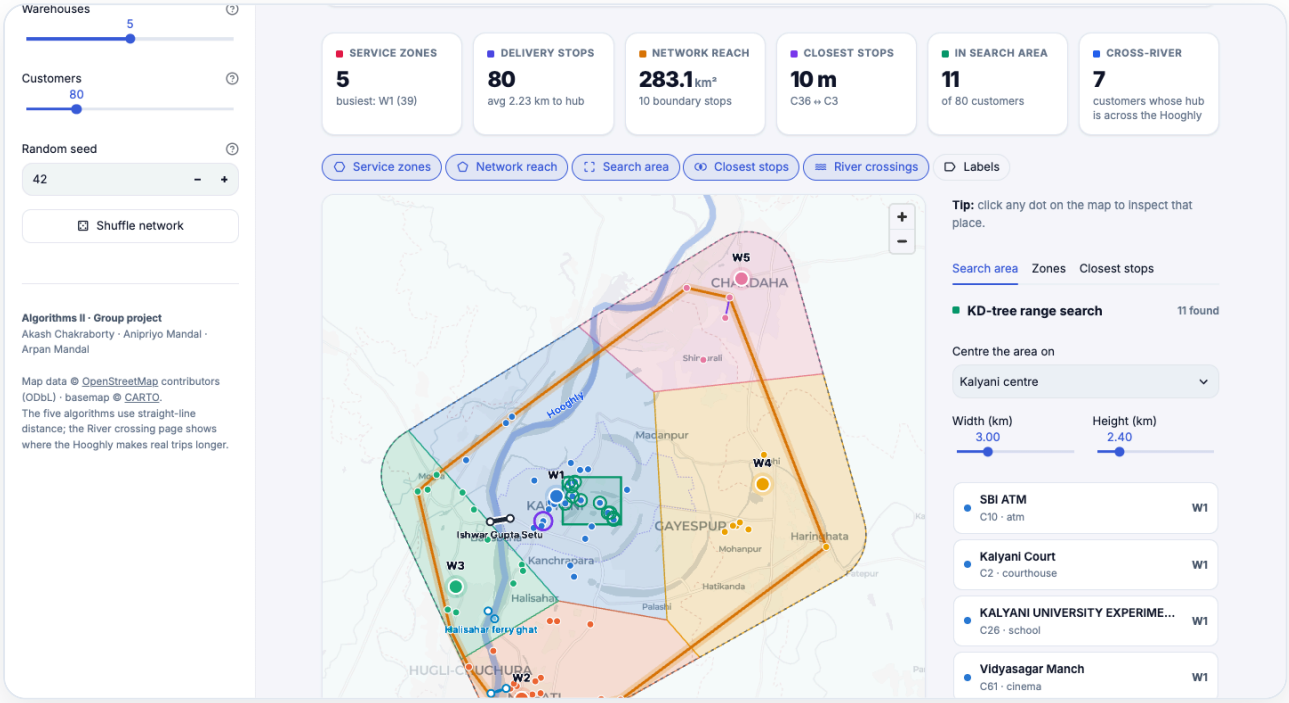
ALGORITHMS II (CSC711) · MINI PROJECT REPORT

# Smart Delivery Network on Kalyani

Computational geometry and approximation algorithms for delivery planning on real OpenStreetMap data, on both banks of the Hooghly

Voronoi · KD-tree · Closest pair · Convex hull · Set cover · River check





**Akash Chakraborty**
Reg. 1059 · Roll CSE/23011

**Anipriyo Mandal**
Reg. 1065 · Roll CSE/23017

**Arpan Mandal**
Reg. 1075 · Roll CSE/23027


**SOURCE CODE**
github.com/anipriyo/smart-delivery-network

**DATE**
22 October 2026

# Contents

1 Problem Statement

2 Input Model

3 Algorithms Used

4 How the Algorithms Combine

5 Dashboard

6 Worked Example (default network)

7 Real-World Issue: Crossing the Hooghly

8 Testing

9 Discussion

10 Data Sources and Attribution

11 Conclusion

A Appendix: Running the Project

**Summary.** A delivery network around Kalyani, West Bengal, built from 571 real OpenStreetMap places in a 401 km² area on both banks of the Hooghly. Five Algorithms II techniques plan it: Voronoi zones (who serves whom), a KD-tree (what is where), divide-and-conquer closest pair (what is near), the convex hull (how far it reaches) and greedy set cover (which sites to keep), shown in an interactive Streamlit dashboard with step-by-step walkthroughs. A river check then measures where straight-line planning breaks: with one road bridge along 27.5 km of river, a van's real trip to a cross-river hub is more than twice the straight line.

# 1. Problem Statement

A delivery company runs several warehouses in and around **Kalyani, West Bengal**, and serves customer stops from Chakdaha in the north to Naihati in the south, on both banks of the Hooghly river. Once the list of stops grows past what a person can eyeball, four questions become hard to answer by hand:

1. Which warehouse should serve which customers?
2. Which stops sit close enough together to be worth grouping?
3. Which customers lie inside a given area (e.g. a planned delivery zone)?
4. How far does the network actually reach, and which sites are worth keeping?

Answering these by comparing every point against every other point takes quadratic time and stops being practical once the stop count grows. Each question is a textbook computational-geometry problem, so each is handed to the algorithm built for it, and the results feed one dashboard drawn on the real street map.

All five algorithms measure distance in straight lines. In this region that assumption has a visible real-world flaw: the Hooghly. A customer on one bank whose nearest warehouse "as the crow flies" is on the other bank can only be reached over a road bridge or by ferry. A sixth, smaller check measures where and by how much the straight-line answers go wrong (§7).

This is a **geometric decision-support tool**, not a full vehicle-routing system: it does not plan turn-by-turn routes, model traffic or fuel, or schedule drivers.

# 2. Input Model

Every warehouse and customer is a point `Point(id, x, y, kind)` with `kind ∈ {warehouse, customer}`. The algorithms only ever read `x` and `y`, which are kilometres on a flat local grid. Each point also carries the real place behind it (`name`, `lat`, `lon`, OSM `category`) for display, and a short label such as `W1` or `C12`.

**The study area.** The area is the convex hull of five town centres, Kalyani, Naihati, Chakdaha, Mogra and Haringhata, padded by 2.5 km. It covers **401 km²**, about 25 × 26 km. Each town's coordinate comes from an OpenStreetMap (OSM) node, except Kalyani's, which is the centroid of the *Kalyani City* outline (OSM way 1094723487).

The Hooghly splits the area into two banks:

- **West bank (80 km²):** Mogra, Bansberia and Hugli-Chuchura.
- **East bank (321 km²):** everything else, including Kalyani, Naihati, Halisahar, Kanchrapara, Haringhata and Chakdaha. Chakdaha is on the east bank because the river curves around Kalyani's north.

27.5 km of the river's main channel runs through the area.

**The points are real places.** The October 2026 extract holds **571 usable named places** inside the area: shops, banks, schools, clinics, offices, college and university buildings, and so on. 128 of them are on the west bank and 443 on the east bank. 52 are tagged as warehouse candidates: markets, supermarkets, post offices, warehouses and industrial land.

**Latitude/longitude is projected to a flat km grid.** An equirectangular projection is anchored at the study area's south-west corner ($\text{lat}_0$ = 22.864°, $\text{lon}_0$ = 88.349°):

```
x = (lon - lon₀) · 111.320 · cos(lat₀)     y = (lat - lat₀) · 110.574
```

so 1 unit is 1 km and every distance can be reported in metres. At this scale the grid is still accurate. Across all 157,880 pairs of places at least 0.5 km apart, the flat-grid distance stays within 0.2 % of the true distance on the WGS84 ellipsoid (worst case 0.15 %). The unit tests check a 9 km diagonal against the haversine great-circle distance to the same tolerance.

**Sampling is seeded.** `sample_network(region, m, n, seed)` picks the network with Python's `random.Random(seed)`, so the same seed always gives the same network:

- **Warehouses** are drawn from the 52 candidates first, then from any place. They must be at least `0.55 · √(area / m)` km apart (4.92 km for m = 5), so no two hubs sit next to each other and the service zones are not degenerate. If the area runs out of far-apart places, the gap shrinks by 20 % and the search repeats.
- **Customers** are a uniform random sample of the remaining places. Places at the exact same spot (to about 1 m) are skipped, so no two customers coincide.

The defaults are seed 42, 5 warehouses and 80 customers. The sidebar allows 2–8 warehouses and 10–300 customers. The cap of 8 lets each zone keep its own colour from a fixed palette that was checked for colour blindness.

**CSV upload.** A user can also upload any network as a CSV with columns `type, lat, lon` and an optional `name` (`type` is `warehouse` or `customer`; at least 1 warehouse, at least 2 customers, at most 8 warehouses). The flat grid is then anchored at the uploaded points' own south-west corner, so any city works, and the zones are clipped to a buffered convex hull of the points. Uploaded networks have no river, so the river check is skipped for them. The Data page exports the current network as a CSV with these columns (plus `label` and `category`), so a downloaded file can be edited and uploaded again.

## 2.1 Data pipeline

The OSM extract is downloaded once by `scripts/fetch_osm.py` and saved in the project, so the algorithms run offline; only the background street-map tiles need internet.

1. **Define the area.** The script builds the padded convex hull of the five anchor towns.
2. **Fetch.** Queries go to the Overpass API for:
   - every named feature inside the area, plus warehouses and industrial land
   - the ways of the OSM river relation "Hooghly" near the area
   - the road bridges, ferry routes and ferry terminals near the area
   - the Kalyani City outline, drawn on the map for context

   Overpass is often overloaded, so the script tries several mirrors and retries with exponential back-off. It downloads only the raw files that are missing; `--refresh` downloads them all again.
3. **Keep the raw responses.** The untouched JSON goes to `data/2026-10/raw/`. Each month gets its own folder and old folders are never edited. The files of the first, Kalyani-City-only extract stay in the same folder as legacy data.
4. **Clean the places.** Each feature is classified by its main OSM tag (amenity, shop, office, healthcare, ...). Roads, rivers, rail lines, parks, parking and plain residential land are skipped, as are duplicates where one place is mapped both as a node and as a building. Warehouse candidates are flagged by tag, and city and town nodes become map labels (10 towns).

5. **Build the river.** The relation's pieces are joined into one main channel. Side channels that loop around islands are dropped, so the line never crosses itself.
6. **Build the crossings (3):**
   - **Ishwar Gupta Setu:** the road bridge between Kalyani and Bansberia, 1.04 km.
   - **Naihati Ferry Ghat:** two OSM ferry terminals on opposite banks, paired, 0.82 km.
   - **Halisahar ferry ghat:** only one terminal is mapped, so its far landing is estimated by mirroring it across the channel, 0.52 km.

   Rail-only bridges (Jubilee and Sampreeti, Naihati–Bandel) are left out because delivery vehicles cannot use them. Ghats the team knows locally can be added by hand in `data/manual/ferries.csv`.
7. **Save.** The outputs are `region.geojson`, `region_places.csv`, `towns.csv`, `river.geojson` and `crossings.csv`. The app always loads the newest complete month.

# 3. Algorithms Used

| # | Algorithm | Question Answered | Technique | Complexity |
|---|---|---|---|---|
| 1 | Convex hull (Andrew's monotone chain, a Graham-scan variant) | How far does the network reach? | Sort points by (x, y), build the lower and upper chains by popping any point that makes a non-left turn | O(n log n) |
| 2 | Closest pair (divide and conquer), region-wide and per zone | Which two stops are nearest each other? | Split by x-median, recurse on both halves, then scan only the thin strip around the dividing line for a closer cross-pair | O(n log n); brute force is $O(n^2)$ |
| 3 | KD-tree + range search | Which customers lie inside this rectangle? | Build a binary tree over customers alternating x/y median splits; a range query skips whole subtrees that cannot meet the query box | Build O(n log n); query $O(\sqrt{n} + k)$ |
| 4 | Voronoi diagram | Which warehouse is nearest to each part of the area? | Cells from scipy/Qhull, clipped to the study area with shapely; each customer is assigned by a direct nearest-site test | Cells O(m log m) expected; assignment O(m) per customer |
| 5 | Greedy set cover (optional extension) | Which few warehouses cover everyone? | Each warehouse covers the customers within a radius; repeatedly keep the warehouse covering the most still-uncovered customers | Exact optimum is NP-hard; greedy is within $H(n) \approx \ln n$ of it |
| + | River check (real-world extension, §7) | Where does straight-line planning break? | Bank test against the river line; trips across go via the best allowed bridge or ferry | O(c) per trip; $O(n \cdot m \cdot c)$ for the whole check |

Here n is the number of customers, m the number of warehouses, k the number of points a query reports and c the number of river crossings.

## 3.1 Convex Hull — Andrew's Monotone Chain

The hull uses **Andrew's monotone chain**, a variant of Graham's scan. Graham's scan sorts the points by polar angle around the lowest point; Andrew's version sorts them by `(x, y)` instead and builds the hull in two halves, which avoids angle comparisons and handles ties simply.

The customers are sorted by `(x, y)` (duplicates removed). The **lower chain** is built by scanning them west to east: each new point is pushed onto a stack, but first the top of the stack is popped for as long as the last two stack points and the new point fail to make a left (counter-clockwise) turn, tested with the sign of a cross product. The **upper chain** is built the same way from east to west. Joining the two chains gives the hull in counter-clockwise order, starting from the westmost stop. Collinear boundary points are dropped. Each point is pushed and popped at most once per chain, so the scan is linear after the O(n log n) sort.

The area the hull encloses is reported as the network's **reach**.

## 3.2 Closest Pair — Divide and Conquer

The customers are sorted by x and split at the median into two halves. Each half is solved recursively (three or fewer points are solved by brute force); let `δ` be the smaller of the two halves' best distances. Any closer pair must have one point on each side and both within `δ` of the dividing vertical line, so only that strip of width 2δ is re-examined. Points in the strip are taken in y order, and each is compared only with the points above it until the y-gap reaches `δ`; a packing argument bounds this to a constant number of comparisons per point.

No level sorts anything. The points are sorted by x once at the start, and every recursive call returns its points sorted by y, built by merging the two halves' y-orders in linear time, as in merge sort. The strip is filtered from that merged list, so $T(n) = 2T(n/2) + O(n) = O(n \log n)$, against $O(n^2)$ for comparing every pair.

The dashboard runs it on all customers (the two stops in the whole region that are nearest each other) and again **inside each Voronoi zone**, giving each warehouse its own pair of stops worth grouping into one delivery.

### 3.3 KD-Tree and Range Search

A KD-tree is built by recursively splitting the customer set at the median x (even depth) or median y (odd depth), so the tree is balanced with depth about $\log_2 n$. The points are sorted once by x and once by y. Each node takes the median of the order for its axis and splits both orders in one linear pass (comparing ranks with the median's), so both stay sorted without re-sorting: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

A rectangle range query recurses from the root. At each node it reports the node's point if it lies inside the box, then visits a child only if the box reaches that side of the splitting line. Whole subtrees whose part of the area the box cannot touch are skipped. A query therefore touches only the points near the box plus the ones it reports, $O(\sqrt{n} + k)$ for a balanced 2D tree, instead of scanning every customer.

### 3.4 Voronoi Diagram

Each warehouse owns a cell of the area: the set of points closer to it than to any other warehouse.

- **Cells.** The cell geometry comes from `scipy.spatial.Voronoi`, which wraps Qhull. Qhull builds the Delaunay triangulation of the sites (as the lower convex hull of the sites lifted onto a paraboloid) and returns its dual, the Voronoi diagram, in O(m log m) expected time. Four helper sites far outside the area are added so every real warehouse gets a closed cell. Each cell is then clipped to the study area with shapely, so the zones match the map and their areas add up exactly to the area's total.
- **Assignment.** Which warehouse serves a customer is decided by a direct nearest-site test: the customer is compared with every warehouse and the nearest one wins. This is O(m) per customer and exact by the definition of a Voronoi cell, so the decision never depends on polygon geometry. The tests check that every customer does fall inside its assigned warehouse's clipped cell.

### 3.5 Greedy Set Cover

Given the warehouses as candidate sites and a coverage radius, each candidate covers the customers within that radius (straight-line). The true minimum number of sites needed to cover every customer is an NP-hard set-cover instance, so a greedy heuristic is used instead: repeatedly keep the candidate covering the most still-uncovered customers, until every customer is covered or no candidate can add anyone new. This greedy rule is proven to be within a factor `H(n) = 1 + 1/2 + ... + 1/n ≈ ln n` of the true optimum. The set-cover example from the project overview (four sites, nine customers, answer W2 then W4) is one of the unit tests.

## 4. How the Algorithms Combine

- **Voronoi + KD-tree.** Voronoi decides which warehouse owns each customer. The KD-tree then pulls out the customers inside any rectangle on the map, and their warehouse labels come along for free, so one query answers both "who is here" and "who serves them."

- **Voronoi + closest pair.** Closest pair is run inside each warehouse's zone to find the stops worth grouping into a shared delivery. This is proximity analysis only; it does not solve routing.
- **Convex hull + everything.** The hull marks the outer edge of the customer set, showing how far the network reaches compared with the whole area. Its corners are the outlying stops, which are also the ones a coverage radius is most likely to miss.
- **Set cover + Voronoi.** Voronoi assigns each customer to exactly one warehouse; set cover asks a different question, which warehouses are needed at all for a given reach.
- **Voronoi + river check.** The river check takes Voronoi's straight-line assignment and tests it against the real geography: wherever a zone spills across the Hooghly, it measures the real trip and the hub that is actually nearest.

## 5. Dashboard

The dashboard is a Streamlit app (`app.py` and `ui/`) with six pages in a top navigation bar. Maps are deck.gl (via pydeck) on CARTO's light "Positron" basemap, which is built from OpenStreetMap data. The app uses a single light theme. Every page has a header with the project name, the course and summary chips; the Network map and How it works pages also list the team.

The **sidebar** chooses the data source (*Kalyani region* or *Upload CSV*), the number of warehouses (2–8) and customers (10–300), and the random seed, with a *Shuffle network* button for a new seed. It also shows the course, the team and the OSM and CARTO credits.

1. **Network map.** The headline cards show:
   - service zones (and the busiest one)
   - delivery stops (and the average distance to their hub)
   - network reach (hull area and number of boundary stops)
   - the closest pair
   - the customers in the search area
   - the customers whose straight-line hub is across the Hooghly

   The map shows the Voronoi zones coloured by warehouse and clipped to the study area, the convex hull, the KD-tree search area with its hits ringed, and the closest pair (region-wide, thick) and per-zone pairs (thin). It also shows the river with its bridge and ferries, and the Kalyani City outline for context; each layer can be switched off. Clicking any dot inspects that place. A warehouse shows how many customers it serves; a customer shows its three nearest warehouses with distances. Either can become the centre of the search area.

   A side panel has three tabs:

   - **Search area:** centre on Kalyani or any place; width and height 0.5–10 km, default 3.0 × 2.4 km; hits listed nearest first.
   - **Zones:** customers, average and farthest distance per zone.
   - **Closest stops:** region-wide and per zone.
2. **River crossing.** The river check of §7, with a switch between *Delivery van* (road bridges only) and *Two-wheeler* (bridges and ferries). The cards show:
   - the west-bank customers

- how many have their hub across the river
- the average straight-line and real trip
- the extra distance
- how many have a nearer hub once the river counts

The map draws each affected customer's straight line next to its real route over the bridge or ferry. A side list ranks the affected customers by extra distance and lists the three crossings.

3. **Coverage planner.** Greedy set cover with a radius slider (1–10 km, default 4 km). The cards show:
   - how many warehouses are kept
   - the share of customers covered
   - how many are not covered
   - the number of rounds
   - the H(n) guarantee

   The map draws each kept warehouse's radius in its zone colour, the warehouses that could close as grey outlines, and rings any uncovered customer in red. A round-by-round timeline lists each pick, how many new customers it covered and the running total.

4. **Walkthrough.** Steps through every decision of five algorithms on the customers currently on the map:
   - **Convex hull:** the chain being built and each popped point.
   - **Closest pair:** each split line, base case and the strip of width 2δ.
   - **KD-tree build:** each median split.
   - **KD-tree search:** each visited node and skipped subtree.
   - **Voronoi assignment:** one customer compared with each warehouse in turn.

   Arrows and a slider move between steps. The traces come from instrumented copies of the algorithms in `src/traces.py`, kept apart from the tested core code so the teaching output cannot change a result; a test checks that they agree.

5. **Data.** The full point set as a table (label, name, type, OSM category, zone, bank, distance to hub, x/y in km, latitude/longitude), with a CSV download, a bar chart of customers per zone and a chart of the kinds of places.

6. **How it works.** One card per algorithm and one for the river check, each with its question and complexity, then the OSM pipeline in four steps and the scope and limits.

# 6. Worked Example (default network)

Input: the October 2026 OSM extract with the app's defaults: **seed 42, 5 warehouses, 80 customers**, the default search area (3.0 × 2.4 km around Kalyani City) and the default 4 km coverage radius. Every number below can be reproduced with `python report/worked_example.py`, which calls the same code as the app.

**Warehouses chosen by the seeded sampler** (no two are closer than 6.64 km)

| Label | OSM place | Category | Bank | x (km) | y (km) |
|---|---|---|---|---|---|
| W1 | More (Kalyani) | supermarket | east | 9.00 | 12.66 |
| W2 | Naihati Post Office | post office | east | 7.21 | 2.35 |
| W3 | Industrial site | industrial | west | 3.83 | 8.06 |
| W4 | Flipkart Warehouse | warehouse | east | 19.60 | 13.27 |
| W5 | Chakdaha Post Office | post office | east | 18.51 | 23.72 |

13 of the 80 customers are on the west bank.

**Results**

| Step | Input | Output |
|---|---|---|
| Voronoi | 5 warehouse sites, 80 customers | 5 zones clipped to the area, areas summing to 400.78 km². Customers per zone: W1 39, W2 18, W3 12, W4 7, W5 4. Average straight-line distance to the assigned hub 2.23 km. |
| Convex hull | 80 customers | **10 boundary stops**, enclosing 283.1 km² (71 % of the study area) |
| Closest pair (region-wide) | 80 customers | **C36 Shivam Enterprise ↔ C3 S B Printers, 10 m** (brute force would compare 3,160 pairs) |
| Closest pair (per zone) | each zone's customers | W1: C36 ↔ C3, 10 m · W4: C50 APC Ray Lecture Hall Complex ↔ C22 Indian Overseas Bank, 185 m · W2: C15 Bankim Pathagar ↔ C25 Katyayani Girls High School, 190 m · W3: C28 Pharmacy ↔ C38 Dey's Residence, 318 m · W5: C24 Palpara ↔ C13 Bapuji Balika Vidyamandir, 1.06 km |
| KD-tree range query | Default search area: 3.0 × 2.4 km centred on Kalyani City's centroid (x 9.32–12.32 km, y 11.22–13.62 km) | **11 customers**, all in zone W1, among them Kalyani Court, the University of Kalyani, Vidyasagar Manch and the A.P.J. Abdul Kalam Auditorium. The search visited 25 of the 80 tree nodes and skipped 8 subtrees; a brute-force filter returns the same 11. |
| Greedy set cover | 5 candidate warehouses, radius 4 km | **5 rounds**: keep W1 (+36, 36/80), W2 (+13, 49/80), W3 (+6, 55/80), W4 (+6, 61/80), W5 (+3, 64/80). No warehouse can close. **16 customers** are more than 4 km from every warehouse (the farthest, C71 Jagorani Football Ground, is 5.77 km away). |

**Step-by-step traces.** For a feel of the work each algorithm does on this input:

- **Convex hull:** 308 steps (160 pushes, 148 pops).
- **Closest pair:** 125 steps (31 divides, 32 base cases, 31 combines and 31 strip checks).
- **KD-tree build:** 80 median splits over 7 levels.
- **KD-tree search:** 33 steps (25 visits, 8 skipped subtrees).

- **Voronoi trace:** customer C1 (Ramchandrapur Health Centre) is 10.87 km from W1, 5.09 km from W2, 10.24 km from W3, 13.18 km from W4 and 22.31 km from W5, so it joins zone W2.

**What the numbers say**

- W1 (More, a supermarket in Kalyani) is the busiest hub, with 39 of the 80 customers. Kalyani holds most of the mapped places, and the whole default search area falls inside W1's zone.
- W4's zone is nearly as large as W1's (117 km² against 119 km²) but has only 7 customers. The east of the area (Gayespur, Haringhata) has few places in OSM. Zone sizes reflect mapping density, not real demand.
- The hull covers 71 % of the study area: the customers are spread over the whole region, not just Kalyani.
- At 4 km every warehouse is needed, because each covers customers no other warehouse reaches, and 20 % of the customers are still out of reach. The hull's corners (C71, C76, C1, …) are among them.
- The closest pair is only 10 m apart, an electrical shop and a print shop at essentially the same spot, so one delivery could serve both.

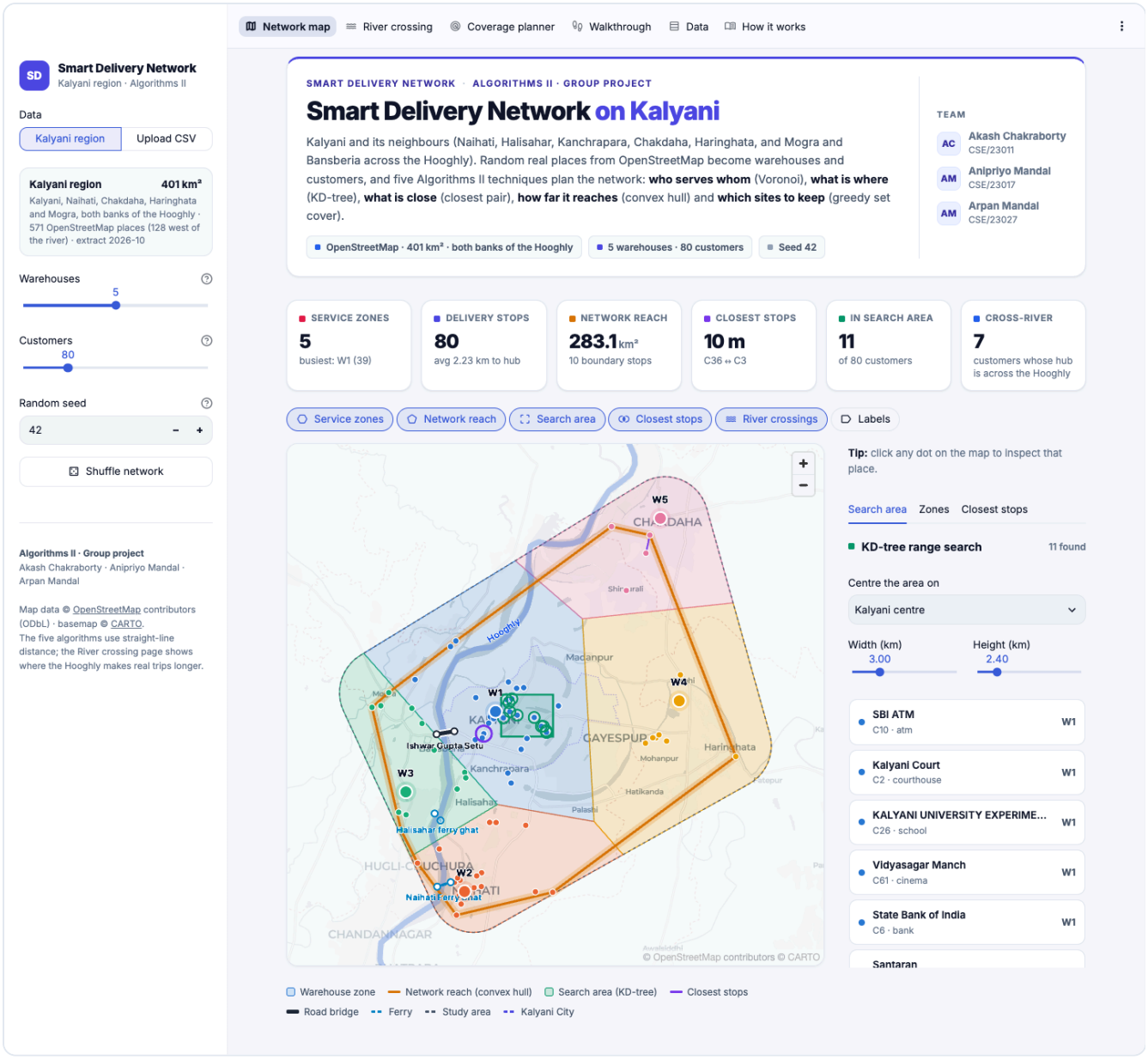



**Figure 1. Network map.** Network map page for the default network: Voronoi service zones coloured by warehouse and clipped to the study area, the convex hull (orange), the KD-tree search area around Kalyani with its 11 hits ringed (green), the closest stops region-wide and per zone (purple), and the Hooghly with Ishwar Gupta Setu and the two ferry ghats. The cards above the map report 5 zones, 80 stops, a 283.1 km² reach with 10 boundary stops, a 10 m closest pair, 11 customers in the search area and 7 customers whose hub is across the river. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

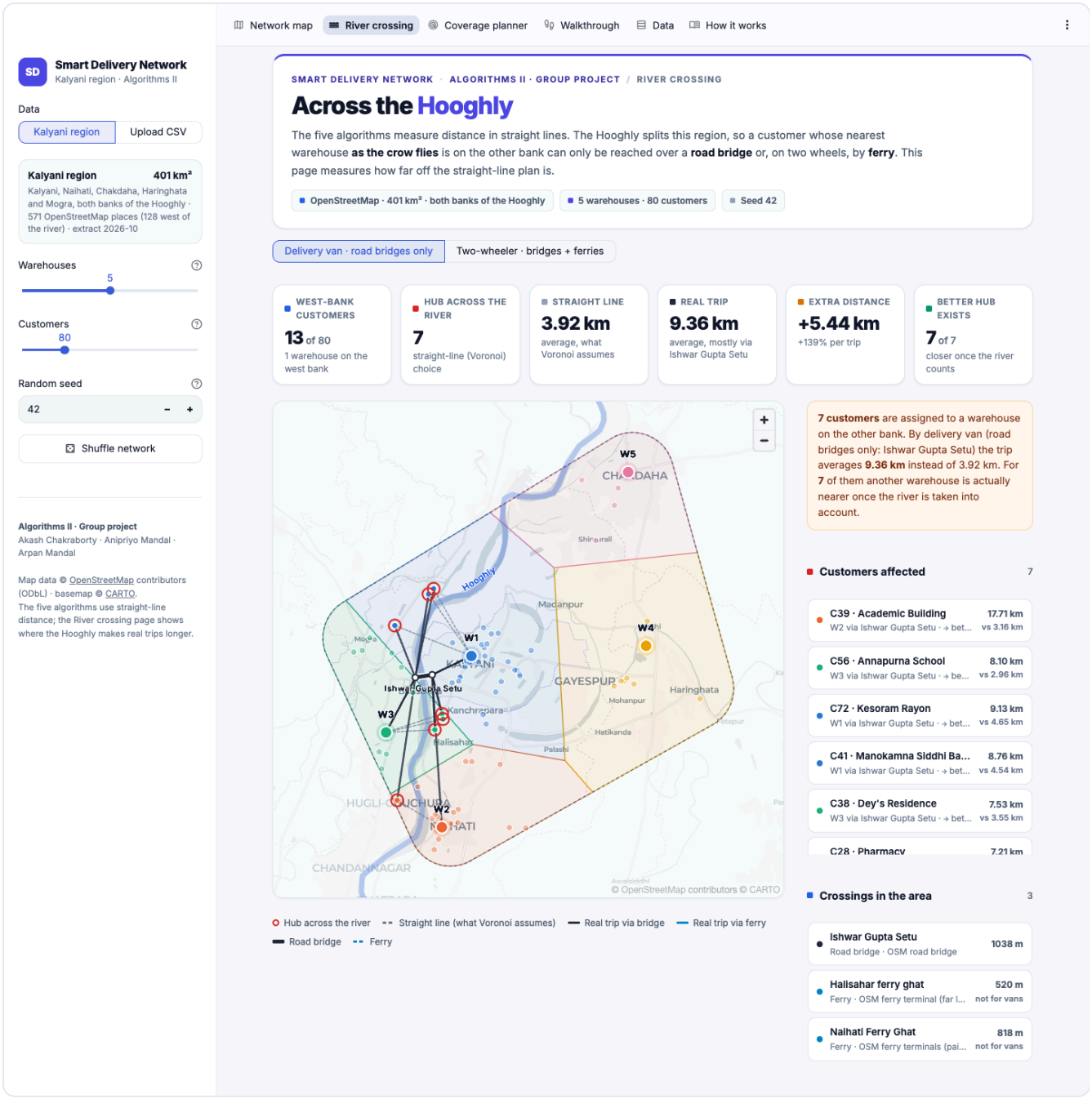



**Figure 2. River crossing.** River crossing page in delivery-van mode: the 7 customers whose straight-line hub is across the Hooghly are ringed in red, with their straight lines (dashed) and their real routes over Ishwar Gupta Setu (solid). The cards report 13 west-bank customers, 7 with a hub across the river, 3.92 km straight against 9.36 km real on average (+139 %), and a nearer hub for all 7. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

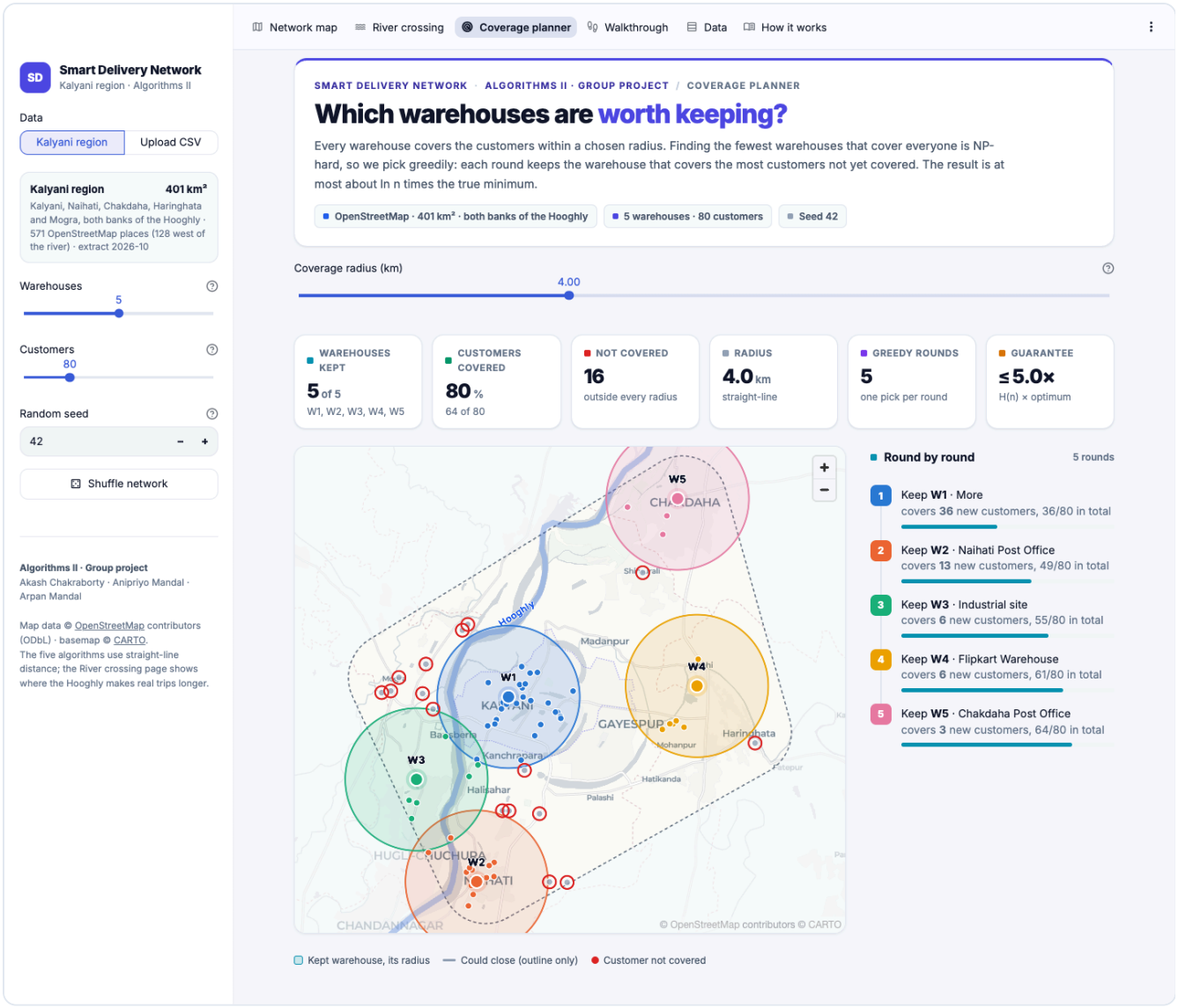



**Figure 3. Coverage planner.** Coverage planner at the default 4 km radius: all five warehouses' radii in their zone colours and the 16 uncovered customers ringed in red. The timeline on the right lists the five greedy rounds (W1, W2, W3, W4, W5) and the running total up to 64 of 80. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

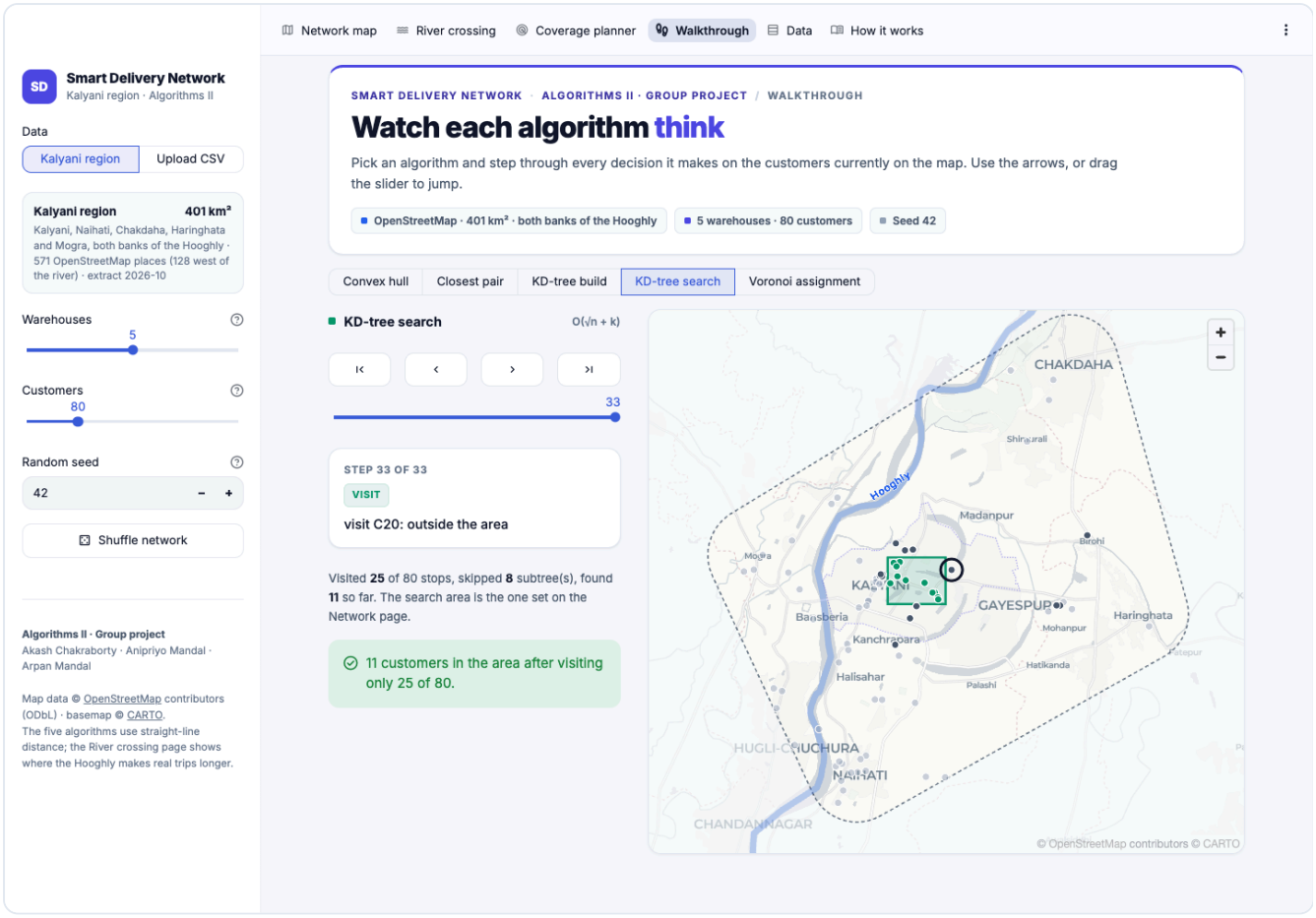



**Figure 4. Walkthrough (KD-tree search, final step).** Walkthrough page, KD-tree search, step 33 of 33: the default search area around Kalyani (green box), the 11 customers found inside it (green), the nodes the search visited (dark grey) and the stops it never had to look at (light grey). The summary reads: visited 25 of 80 stops, skipped 8 subtrees, found 11. Map data © OpenStreetMap contributors (ODbL), basemap © CARTO.

# 7. Real-World Issue: Crossing the Hooghly

Voronoi assigns each customer to the warehouse nearest **in a straight line**. Across a river that is wrong: the trip has to use a crossing. The river check ( `src/river.py` ) measures how wrong.

**Model.**

- **Banks.** The river's main channel is extended at both ends past the study area and used to cut the area into two pieces with shapely; the piece whose centroid lies further west is the west bank. A point's bank is the piece that contains it. Points that fall in neither piece are decided by a cross-product side test against the nearest stretch of the channel.
- **Trips.** A trip between two points on the same bank is the straight line. Between banks it is the shortest `|p - landing on p's bank| + crossing length + |landing on q's bank - q|` over the allowed crossings, which is O(c). A **delivery van** may use only road bridges (here just Ishwar Gupta Setu); a **two-wheeler** may also take the two ferries.
- **Check.** For every customer whose straight-line (Voronoi) hub is on the other bank, it records the straight-line distance, the real trip and the crossing used. It then finds the hub that is really nearest once the river counts, by routing the customer to every warehouse, which is O(m · c) per customer.

**Results on the default network** (13 customers and one warehouse, W3, are on the west bank):

| | Delivery van (road bridges only) | Two-wheeler (bridges + ferries) |
|---|---|---|
| Customers whose straight-line hub is across the river | 7 | 7 |
| Average straight-line distance (what Voronoi assumes) | 3.92 km | 3.92 km |
| Average real trip | 9.36 km | 6.32 km |
| Extra distance per trip | **+5.44 km (+139 %)** | **+2.40 km (+61 %)** |
| Crossings used | Ishwar Gupta Setu × 7 | Ishwar Gupta Setu × 3, Halisahar ferry × 3, Naihati ferry × 1 |
| A nearer hub exists once the river counts | 7 of 7 | 5 of 7 |

- **The worst case is C39 (Academic Building), on the west bank.** Voronoi gives it to W2 in Naihati, 3.16 km away across the water. By van the only way is up to Ishwar Gupta Setu and back down, 17.71 km, while W3 on its own bank is 4.14 km away. By two-wheeler the Naihati ferry makes it 3.54 km, so W2 is the right hub after all.
- **Some of the "nearer hub" cases are marginal.** For C72 (Kesoram Rayon) and C41 the van trip to the Voronoi hub W1 is 9.13 km and 8.76 km. W3 is only 10 m and 40 m shorter. So the river matters most for customers near the water, far from the only road bridge.

In short, with only one road bridge along the 27.5 km of river inside the area, the straight-line service zones are wrong for about 1 in 11 customers when deliveries go by van. For two-wheelers, the ferries cut the extra distance by more than half.

# 8. Testing

There are **36 automated tests** in three files, and all pass ( `python -m pytest tests/ -v` ).

`tests/test_algorithms.py` (19 tests) checks each algorithm against a brute-force reference or a known answer:

- Closest pair matches an $O(n^2)$ pairwise scan over five random seeds and many sizes, handles duplicate coordinates, and passes a small hand-made case. The y-merge is sorted and stable, and a test fails if the recursion ever sorts again after the single initial sort by x (the O(n log n) guarantee).
- Convex hull keeps only the corners of a square with an interior point, and handles a fully collinear point set (degenerate case).
- KD-tree range queries match a brute-force filter (also with many duplicate coordinates), and an empty box returns nothing. The tree keeps the split invariant and stays balanced when values tie, and a test fails if the build sorts anything beyond the two initial presorts.
- Greedy set cover covers every coverable customer, reproduces the overview document's worked example (W2 then W4), and stops when no candidate adds anything new.
- The Walkthrough traces agree with the core algorithms (hull, closest pair, KD-tree search), also on duplicate points, and the Voronoi trace ends at the nearest warehouse.

`tests/test_geo.py` (10 tests) checks the geography and data handling:

- The projection round-trips latitude/longitude, and a 9 km diagonal agrees with the haversine distance to within 0.2 %.
- The OSM extract is sane: an area of 300–500 km², at least 400 places, at least 50 of them on the west bank, all inside the area.
- Sampling is reproducible for a fixed seed, never puts a customer on a warehouse or two customers on the same spot, and keeps warehouses more than 2 km apart.
- For 1, 2, 5 and 8 warehouses, the clipped Voronoi zones partition the area (their areas sum to the area's total) and every customer lies inside the zone of the warehouse it was assigned to (4 tests).
- A network survives a CSV export and re-import, and a CSV with missing columns is rejected.

`tests/test_river.py` (7 tests) checks the river model, mostly on a toy 10 × 10 km square split by a north–south river with one bridge and one ferry:

- The two banks are found, and every crossing is oriented west to east.
- A same-bank trip is the straight line.
- A cross-river trip uses the best allowed crossing (the two-wheeler takes the ferry; the van must use the bridge and goes further).
- With no allowed crossing the trip is impossible (infinite).
- The river check flags only cross-bank assignments and finds the better hub.
- Joining river pieces gives one simple line past an island.
- On the real data, Mogra is on the west bank and Kalyani, Naihati, Chakdaha and Haringhata are on the east. Ishwar Gupta Setu is a bridge, every crossing joins the two banks, and no real trip is shorter than its straight line.

# 9. Discussion

## Limitations

- Euclidean (straight-line) distance stands in for road distance. Apart from the river check, there are no roads, one-way streets or traffic.
- The river check is itself simplified:
  - It is still straight-line on each bank.
  - The OSM channel geometry is coarse (vertices up to 1.5 km apart), so places right on the bank can land on the wrong side.
  - The Halisahar ferry's far landing is estimated.
  - Ferries have no timetables or waiting times.
  - Only three crossings are known in OSM for this stretch of the river.
- The system does not plan a route. It identifies structure (who serves whom, who is nearby, how far the network reaches) rather than an ordered path for a vehicle to drive.
- The customers are random real OSM places standing in for customers, not real orders. OSM coverage is uneven: places cluster in Kalyani, and the east and north are sparse, so some zones get few customers (W4 and W5 in the example above).
- Greedy set cover gives an approximate, not optimal, warehouse subset; for instances this small an exact solver would also be feasible.
- The Voronoi cells come from scipy/Qhull, not from Fortune's sweep-line algorithm named in the project overview. Both are O(m log m), but the sweep itself is not implemented or shown in the Walkthrough.
- The flat projection is only valid at regional scale; an uploaded CSV that spans a whole country would distort distances.

## Future Scope

- Replace straight-line distance with road-network distance, e.g. shortest paths over the OSM street graph, which the same Overpass pipeline can download. The river would then be handled automatically.
- Make the assignment river-aware: assign each customer by real trip length instead of straight-line distance (a Voronoi diagram under a different distance), and let the Coverage planner use the same distance.
- Add the ferry ghats the team knows locally to `data/manual/ferries.csv`.
- Extend from static demand (all customers must always be served) to **capacitated** facility assignment, where each warehouse has a maximum number of customers it can serve, and weight customers by expected demand.
- Combine the Voronoi/KD-tree layer with an actual routing stage (e.g. nearest-neighbour or 2-opt TSP heuristics) per warehouse zone.
- Implement Fortune's sweep for the Voronoi diagram and add it to the Walkthrough.

## 10. Data Sources and Attribution

- **Map data © OpenStreetMap contributors**, available under the Open Database License (ODbL), https://www.openstreetmap.org/copyright. The places, towns, river, crossings and the Kalyani City outline were downloaded through the Overpass API in October 2026 and are stored in `data/2026-10/`.
- **Basemap tiles © CARTO** (https://carto.com/attributions), the light "Positron" style drawn from OpenStreetMap data, loaded by the dashboard's maps.
- Software: Streamlit, pydeck/deck.gl, SciPy (Qhull), shapely, pandas, Altair and pytest.

## 11. Conclusion

Voronoi decides who serves whom, the KD-tree finds what is where, closest pair finds what is near, the convex hull shows how far the network reaches, and greedy set cover picks the sites worth keeping. Run on real OpenStreetMap places around Kalyani and drawn back on the street map, they turn a list of warehouse and customer coordinates into an interactive, geometrically grounded view of the delivery network, each piece solved by the Algorithms II technique it fits.

The river check shows where that straight-line view stops being enough. With one road bridge along 27.5 km of the Hooghly, a van's real trip to a cross-river hub is more than twice the straight line. Knowing this is the first step towards distance models that follow the real geography.

# Appendix A. Running the Project

The complete source code, data and tests are in the GitHub repository https://github.com/anipriyo/smart-delivery-network. Python 3.12 was used.

```
git clone https://github.com/anipriyo/smart-delivery-network.git
cd smart-delivery-network
pip install -r requirements.txt
streamlit run app.py
```

The dashboard opens at http://localhost:8501. The OpenStreetMap extract is part of the repository ( `data/2026-10/` ), so the algorithms need no download; only the background street-map tiles need internet.

| Task | Command |
|---|---|
| Run the 36 tests | `python -m pytest tests/ -v` |
| Re-download the OSM extract (current month) | `python scripts/fetch_osm.py` |
| Print every number used in this report | `python report/worked_example.py` |
| Retake the four screenshots (app running) | `python report/take_screenshots.py` |
| Rebuild this PDF from `report/report.md` | `python report/build_pdf.py` |

## Repository layout

| Path | Contents |
|---|---|
| `app.py` , `ui/` | Streamlit dashboard: pages, deck.gl map layers, styling |
| `src/` | The algorithms ( `convex_hull.py` , `closest_pair.py` , `kdtree.py` , `voronoi.py` , `set_cover.py` ), the river model ( `river.py` ), geography and sampling ( `geo.py` , `data.py` ), Walkthrough traces ( `traces.py` ) |
| `tests/` | `test_algorithms.py` , `test_geo.py` , `test_river.py` |
| `scripts/fetch_osm.py` | OpenStreetMap download and cleaning |
| `data/` | The October 2026 extract and hand-added ferries |
| `report/` | This report, its screenshots and the scripts that produce them |
| `Smart_Delivery_Network_Overview_Document.pdf` | The project overview the work started from |